

HostFS - the host file system as the boot path

HostFS: a host directory as a RISC OS disc

How the RISC OS Pi 4 emulator gained a filing system whose files are ordinary files in a folder on the Mac or the PC — and how that folder became the disc the machine boots from, with no SD card at all.

HostFS is three pieces working together. A RISC OS module registers a filing system with FileSwitch. A new QEMU device, `vmchannel`, acts as a doorbell. And a host service inside that device does the real work against a directory. The rule that shaped every decision is written at the top of its design:

Do as much work in the host as possible, and as little in the guest as possible.

Three days of work took HostFS from a first file transfer to a filing system in the ROM that boots the desktop from a folder. This document follows that path, and records where it went wrong along the way.

These documents

Chapter	What it covers
1. Why a host filing system	the network and USB-stick routes, and the shape chosen
2. The doorbell	the vmchannel device, its request block, and where it lives
3. Speaking FileSwitch	a filing system that finally streams, and one data path
4. Reaching guest memory	the host walks the MMU; the 252-byte mystery

These documents, continued

Chapter	What it covers
5. Names, types and dates	, xxx suffixes, a generated type table, character sets, time
6. Into the ROM	why a filing system cannot load itself, and splicing a stock ROM
7. Booting from a directory	boot, CMOS on the share, modules on the share
8. Measured, limited, unfinished	SD card against HostFS, the open list, the risks

At a glance

Filing system	HostFS, number 220, registered with FileSwitch
Transport	vmchannel: a synchronous MMIO doorbell; one crossing per operation
Module	2.02, 10,300 bytes, runs from ROM or soft-loaded
Boot	a stock ROM spliced with HostFS; !Boot and CMOS on the share
Measured	boot 14.4 s against 15.7 s from the card; 2,657 doorbells against 2.4 M register accesses
Code	hw/misc/vmchannel.c, riscos-pi4/hostfs/dde, riscos-pi4/tools/mkrom.py

Three layers



This document describes the fork as published at [e7e6deffae](#), 13 September 2026. The [QEMU A72 walkthrough](#) covers the emulated machine, and the [fake it in software walkthrough](#) covers the design principle behind the doorbell.

1. Why a host filing system

A developer using the emulator needs to move files in and out of the guest all the time: a freshly compiled module, a source file edited on the host, a log. The SD card image is a poor way to do that. It is a disc image, so files can only be written into it with the machine stopped, and every change means re-attaching the image. This chapter covers the three routes that were considered, why two of them were dropped, and the shape of the one that was built.

Route 1: the network, and a dialect gap

RISC OS ships an SMB client, LanManFS. QEMU's user-mode network puts the host at `10.0.2.2`. If LanManFS could mount a Windows share, host files would arrive with no guest code at all. That was worth a day to find out.

Two ends that cannot agree

It was ruled out from both ends. The guest reached the host fine: NetSurf browsed through the network, and a ping to the host answered in 2 ms. But:

Side	What it speaks
the ROM's LanManFS	exactly two dialects, both SMB1; no SMB2 or SMB3 dialect exists in the ROM
Windows 11 on the build machine	SMB1 disabled, and signing required

`*LMount` brought up the login dialogue, and the logon failed as *Bad authentication*, which is how LanManFS reports a refused negotiation. Re-enabling SMB1 on the host would have worked, but it is a deprecated protocol with signing turned off. The record calls that "not recommended, not the answer".

Route 2: a USB stick made of a folder

QEMU's `vvfat` block driver can present a host directory as a FAT-formatted disc. Attached as a USB mass-storage device, it would look to RISC OS like a USB stick. That route needs no guest module at all.

The experiment on the Mac got partway. The ROM's USB and SCSI stack claimed the device far enough to run the bulk-only transport: three bus resets, a LUN probe, and one SCSI `INQUIRY` per reset. Then it stopped. There was no *test unit ready*, no *read capacity* and no sector reads. The stick was visible on the bus and never mounted. The note records the likely cause as either something in QEMU's `INQUIRY` reply that the driver rejects, or a class driver that only a full boot loads. It lists the next steps, and none of them was recorded as done. HostFS made the route unnecessary.

Route 3: a doorbell and a module

The plan that was built is the pattern RISC OS emulators already use. VirtualAcorn and RPCEmu both ship a `HostFS:` whose module talks to the emulator. The difference here is the transport: a small device, rather than intercepted SWIs. No CPU state is forged, and any guest code could use the device.

The three layers

The design has three layers:

Layer	Where	Job
the HostFS module	the guest, registered with FileSwitch as filing system 220	turn each FileSwitch call into one request
<code>vmchannel</code>	a QEMU MMIO device at a hole in the Pi 4 memory map	the doorbell: one write carries a request to the host
the host service	inside the device, in QEMU	do the filing-system work against a host directory

The governing rule

The second version of the design, written after the first had been measured and found wanting, opens with the rule quoted on the index page: *as much work in the host as possible, as little in the guest as possible*. It has concrete consequences, and every later chapter is one of them:

- **The wire speaks FileSwitch, not POSIX.** The module copies RISC OS registers into a request and back out, without interpreting them. The host decodes them.
- **The host reaches guest buffers itself**, by walking the guest's page tables, instead of the guest translating addresses (chapter 4).
- **The host owns names, types and dates**, including the RISC OS load and execute words (chapter 5).

- **The guest module stays small.** The design targeted a few hundred lines. The built module is larger, at 1,724 lines of C plus 507 of assembly, and chapter 8 says why.

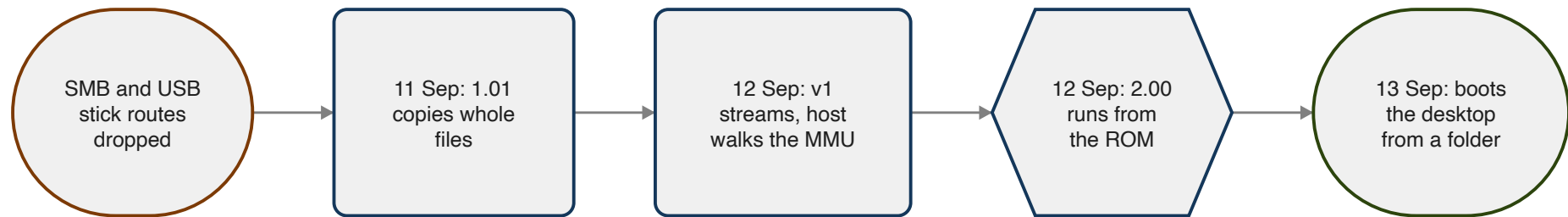
The timeline

When	Commits	What happened
10 Sep	af5a2cbcd1	the SMB route shown dead
10 Sep	99abaa1b9c, d034d7a17a	the design, and the device with a bare-metal smoke test
11 Sep	0a63382fb3 ... 8eb8209624	the doorbell mirrored where the guest can reach it; file transfer end to end: version 1.01
12 Sep	c5a13d181f ... 174a719248	version 1: streams, the host walking the MMU, names and types

The timeline, continued

When	Commits	What happened
12 Sep	bb79635c18	HostFS 2.00 runs from a spliced stock ROM
13 Sep	cf1e8da677, 509ab9d478	the machine boots from the share; CMOS persists on it
13 Sep	676e664065 ... e7e6deffae	2.01 names every error; names Windows cannot store; 2.02 pages and sorts listings

From a folder to a boot disc



2. The doorbell

Every HostFS operation crosses from the guest to the host through one small QEMU device, `vmchannel`. This chapter walks through it: the registers, the request block, why every command runs synchronously, and why the device ended up in two places in the memory map. It also covers the module code that rings it, and a struct bug that left every error blind.

Nothing asynchronous, on purpose

The first design states the shape in one paragraph. The guest writes a request block into its own RAM, pokes the doorbell with the block's physical address, and reads a status register. QEMU executes the command **synchronously, inside the MMIO write, under the big QEMU lock** — the same way the framebuffer's mailbox tags are handled.

The design is candid about the trade:

A slow host disc stalls the vCPU for the duration, which is exactly what a slow disc does on real hardware; in exchange there is no ring, no interrupts, no reorder window, nothing to migrate beyond four registers, and no way for guest and host views of a request to disagree.

The registers

The device is a 16 KiB region with five registers:

Offset	Register	Meaning
0x00	MAGIC	reads 'VMCH', so a module can probe safely
0x04	VERSION	the protocol version, 1
0x08	FEATURES	file operations (only with a share), console, time, v1 frames, logical addresses
0x0c	CMD	write the request block's physical address: this write is the doorbell
0x10	STATUS	always reads 1, because the work is already done

The write handler is the whole transport:

```
static void vmchannel_write(void *opaque, hwaddr offset, uint64_t value,
                           unsigned size)
{
    VMChannelState *s = VMCHANNEL(opaque);

    vmch_trace("vmch WR off=%llx sz=%u val=%08llx\n",
               (unsigned long long)offset, size, (unsigned long long)value);

    if (offset == VMCH_CMD && size == 4) {
        vmchannel_do(s, (hwaddr)(uint32_t)value & ~(hwaddr)0xf);
    }
    /* every other register is read-only */
}
```

`FEATURES` sets its file-operations bit only when a share root was configured, with `-global bcm2838-peripherals.vmchannel-root=DIR`. Without a root the device still answers, so a module can probe for it. The launch scripts set the root from the `RISCOS_HOSTFS` environment variable.

The request block

A request is one page of guest RAM: a 64-byte header, then up to 4,032 bytes of inline data such as a path.

Offset	Field	v0 meaning	v1 meaning
+0	cmd	the command	a FileSwitch entry, 0x100 and up
+4	seq	a sequence number, for the trace	the same
+8	rc	the result code	the same
+12	handle	a file handle, or open flags	a RISC OS error number
+16	arglen	bytes of inline data	the same
+20–+51	scratch	per-command words	R0 to R7 , verbatim

There are two generations of commands in one device. **Version 0** is POSIX-shaped: `OPEN`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `CAT`, `DELETE` and friends, with physical addresses. **Version 1** carries a RISC OS register frame. The module copies R0 to R7 in, rings, and copies them back out, without interpreting them. The device implements both side by side, so card images carrying the old module keep working. Chapter 3 explains why version 1 exists.

One struct, so the two sides cannot drift

The module mirrors the header as one byte-accurate C struct:

```
/* The request block, mirroring include/hw/misc/vmchannel.h exactly.
 * The device reads BYTE offsets; the previous H_* word indices
 * mismatched every field past CMD, so rc/arglen/handle were read from
 * bytes the device never writes. One struct, byte-accurate, so the
 * two sides cannot drift again. */
struct vmreq {
```

```

uint32_t cmd;           /* +0 VMCH_HDR_CMD */
uint32_t seq;           /* +4 VMCH_HDR_SEQ */
uint32_t rc;            /* +8 VMCH_HDR_RC */
uint32_t handle;        /* +12 VMCH_HDR_HANDLE */
uint32_t arglen;        /* +16 VMCH_HDR_ARGLEN */
uint8_t  scratch[44];   /* +20 VMCH_HDR_ARG .. +63 */
uint8_t  data[VMCH_MAX_ARG]; /* +64 VMCH_HDR_SIZE */
};

```

The comment records a real bug from the first day. The module had described the header with *word* indices, which matched the device's *byte* offsets only for the first field. Every later field was read from bytes the device never wrote, so the result code came from the wrong place, and **every error was blind** (commit [0a63382fb3](#)).

Ring the doorbell

On the module side, one small function sends every request. It poisons the result code first, so an unanswered request stays visible instead of a stale success. And the write to `CMD` returns only after the host has done the work, so reading `rc` on the very next line is safe:

```

static uint32_t vmch_go(struct hostws *ws)
{
    uint32_t cmd = req->cmd, r0 = REQW(0);

    req->seq = ++ws->w_seq;
    req->rc = 0xFFFFFFFFu;           /* poison: an unanswered request stays visible */
    vmch_reg(VMCH_CMD / 4) = req_phys; /* the device DMAs physical, not logical */
    if (req->rc != RC_OK) {
        ...                         /* remember the refused request, for the error */
    }
    return req->rc;
}

```

Proving it without RISC OS

As with the mailbox fix in the QEMU walkthrough, the first test used no RISC OS at all. `tools/vmctest.s` is a 196-byte bare-metal program. It rings the doorbell with a ping and prints what came back over the serial port:

Printed	What it proves
48434d56	the magic: the guest found the doorbell
00000000	a poisoned result code was cleared by the device
48434d56	a scratch word written by the device: device-to-guest writes work
11223344	the argument bytes echoed: both directions of memory access work
00000001	STATUS

Where the device lives, and where it had to move

The design needed a physical address that RISC OS's HAL does not name and QEMU does not map. RISC OS on the Pi reads no device tree, so a hole the HAL does not name simply does not exist as far as it is concerned. A survey found one: `0xFD400000`, 20 MiB into the BCM2711's low peripheral window. Nothing in the HAL's device table lives there.

The bare-metal test reached it. **RISC OS did not.** The module's accesses never arrived at the device. The reason is how RISC OS builds its memory map: it maps peripherals on demand, with `OS_Memory 13`, only for the pages the HAL asks for. An address nothing asked for has no logical mapping.

Commit `0a63382fb3` found a place the guest *does* reach. The system timer at `0xFE003000` sits in a 1 MiB section that RISC OS has provably mapped as device memory. So the doorbell is mirrored at `0xFE005000`, an unused hole in that same section:

```
/* RISC OS's logical map is built on demand and the FE00 section
 * (system timer at 0xFE003000) is provably live device mapping.
```

```

* The doorbell therefore also lives at 0xFE005000: an unused hole
* in that same section, so the guest's translations there are real
* device mappings. (An alias at 0xFEC00000 was bypassed by the
* guest: reads there came back right without reaching the device.)
*/
memory_region_init_alias(&s->vmchannel_mr_alias, OBJECT(s),
                        "vmchannel-high",
                        sysbus_mmio_get_region(
                            SYS_BUS_DEVICE(&s->vmchannel), 0),
                        0, 0x1000); /* one page: a 0x4000 alias
                                   * here would shadow the
                                   * hardware at 0x7E007200 */
memory_region_add_subregion(&s_base->peri_mr, 0x5000,
                          &s->vmchannel_mr_alias);

```

The alias is **one page**, not the full 16 KiB. A larger alias would have shadowed real hardware two pages further on (commit [00a61e5e7a](#)). The later blitter device went the other way: it sits in the low window beside the original doorbell, and its module maps that exact page with `OS_Memory 13`.

Mapping on first use

The module maps the doorbell page itself, and it does so **lazily**, on the first filing-system call. `OS_Memory 13` fails when called from a module's initialisation, because the IO area is off limits while a module is loading. The same call works from any ordinary context. The first version reported *OS_Memory 13 failed* at load time until this was understood:

```

static _kernel_oserror *ensure_vmch(struct hostws *ws)
{
    ...
    if (vmch) {
        return NULL;
    }
    r.r[0] = 13; /* OS_Memory: MapIOPermanent */
    r.r[1] = (int)VMCH_PHYS;
    r.r[2] = (int)VMCH_SIZE;
}

```

```
e = swix(XOS_Memory, &r, &r);
if (e != NULL || r.r[3] == 0) {
    return &err_nomap;
}
v = (volatile uint32_t *)r.r[3];
if (v[VMCH_MAGIC / 4] != VMCH_MAGIC_VALUE) {
    return &err_badmagic;
}
if (!(v[VMCH_FEATURES / 4] & VMCH_FEATURE_FS)) {
    return &err_nofs;
}
if ((v[VMCH_FEATURES / 4] & (VMCH_FEATURE_FSENTRY | VMCH_FEATURE_VIRTADDR))
    != (VMCH_FEATURE_FSENTRY | VMCH_FEATURE_VIRTADDR)) {
    return &err_oldhost;
}
...                /* translate the request page once, OS_Memory 0 */
vmch = v;
return NULL;
}
```

The order of the checks is deliberate:

1. **The pointer is published only after magic and features validate.** A mapping onto the wrong page can never be cached.
2. **Each failure has its own error:** no mapping, bad magic, no file support, or an old host.
3. **Version 1 is required, not preferred.** The version 0 transfer path translated addresses in the guest, and fell back to the identity mapping when that failed, writing file data wherever that landed. The source comment's verdict: falling back to it silently would be worse than not loading.
4. **The request block is one page-aligned page,** so a single logical-to-physical translation covers every byte the device reads.

This laziness turned out to matter again later. When HostFS moved into the ROM, its initialisation runs even earlier in the boot, and mapping on first use is what lets it work at all (chapter 6).

Every access, traced

The device writes every MMIO access and every command to a trace file, flushed line by line. Trace lines carry the command, result, path and, since commit `509ab9d478`, the host time the request took. That trace is how the boot was measured in chapter 8.

INFERRED: the trace has no off switch. One recorded instance's trace file is 31 MB. That is fine for a development tool and worth a flag before it ships.

3. Speaking FileSwitch

RISC OS filing systems do not implement `open`, `read` and `write` directly. They register a set of entry points with **FileSwitch**, the layer that sits under every file SWI. FileSwitch calls those entries with register frames, in the shapes the filing system declared when it registered. This chapter covers how HostFS declared itself wrongly at first, the stream bug that resulted, the fix that made it a buffered filing system, and the decision to keep only one data path.

Version 1.01: a file copier

The first module that worked end to end, on 11 September, was a whole-file transfer service. The private repository's record of it is blunt:

Operation	Version 1.01
list directories, copy whole files on and off, delete	worked
stream bytes on an open handle: BGET, OS_GBPB	failed
file types	every file listed as Text, undated
*SetType	reported success and changed nothing
*Rename	failed with <i>Bad rename</i>

That version shipped on the card images the team uses, and it is still there. The emulator keeps its version 0 commands so those images keep working.

The stream bug, found in the source

On 12 September a measurement showed how far version 1.01 was from a real filing system. Every entry in a `*Ex` listing read `FFFFFFFF`. `*Type` printed nothing, and `BGET#` returned garbage before killing the handle.

The cause was a contradiction in what the module told FileSwitch (commit `c5a13d181f`). When a file is opened, a filing system returns an *information word* describing the stream. HostFS set **bit 28**, which declares *unbuffered block transfers*: FileSwitch should send multi-byte transfers to the filing system's own block-transfer entry. But HostFS had left that entry zero in its registration. So every transfer was dispatched into nothing.

Three interfaces, one module

It was worse than one wrong bit. The commit found three interfaces in play:

Interface	What the module did
unbuffered single-byte GetBytes: one byte in R0, C set at end of file	declared, by setting bit 28
buffered GetBytes: buffer, count and offset in R2–R4	implemented , in the handler
the block-transfer entry	left zero

The module had declared one interface, implemented a second, and nulled the third.

The fix: be a buffered filing system

Clearing bit 28 and returning a buffer size makes FileSwitch drive the handlers that already existed, and the handlers were already the right shape. The open handler's exit registers are now:

```
r->r[0] = (int)((1u << 31) | (1u << 30) |
              ((type == 0) ? (1u << 29) : 0u)); /* 29: directory */
r->r[1] = (int)h;
r->r[2] = HOSTFS_BUFSIZE;
r->r[3] = (int)((type == 0) ? 0 : size);          /* extent */
/* allocated space: must be a multiple of the buffer size */
r->r[4] = (int)((size + (HOSTFS_BUFSIZE - 1u)) & ~(HOSTFS_BUFSIZE - 1u));
```

The exit registers

Register	Value
R0	the information word: readable, writable, bit 29 for a directory; bit 28 clear
R1	the host's file handle, 1-based, because 0 means <i>no file</i> to FileSwitch
R2	the buffer size: 1,024 bytes
R3	the file's extent
R4	the allocated size, rounded up to the buffer size

FileSwitch requires the buffer size to be a power of two between 64 and 1,024. HostFS takes the maximum, and the source gives the reason in one line: **the doorbell, not the copy, is what a transfer costs**. Fewer, larger blocks mean fewer crossings.

Being buffered changes three things for the module:

- **Buffered GetBytes has no exit registers**, so the only way to report a failure is to return an error.
- **The last block of a file routinely runs past its end**. The count is a whole number of buffers. The host zero-fills the tail and still delivers the full count.
- **FileSwitch tracks the file pointer and extent itself**.

Verified on the emulator with the module rebuilt inside the guest: `*Type` printed the file, `BGET#` returned the right bytes, a 256 KiB `OS_GBPB` completed, and a C program compiled from HostFS, linked and ran.

One data path, not two

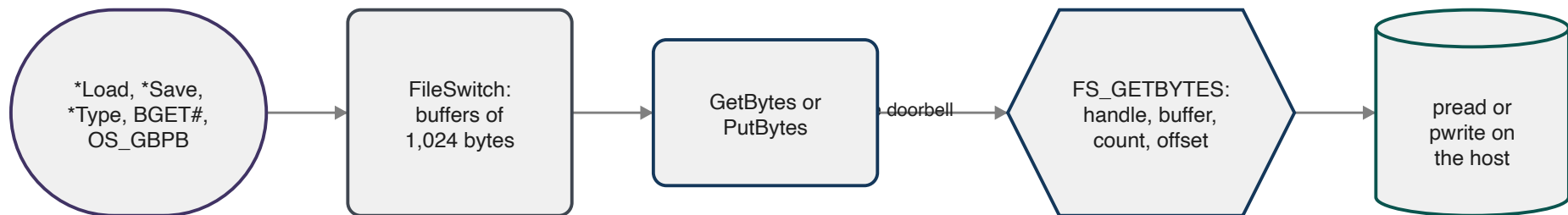
A second change removed the conditions for the bug altogether (commit `1690fac252`). A filing system can let FileSwitch build `*Load` and `*Save` out of open, read, write and close, instead of providing whole-file entries of its own. RISC OS's own guide to writing a simple filing system recommends exactly that for a filing system of this shape. HostFS now sets the two information-word bits that ask for it.

The commit explains why this mattered in one sentence:

Whole-file and streaming were two paths that could disagree, and they did: *Copy worked for months while *Type and BGET# returned garbage, because only the whole-file path was ever exercised.

The retired whole-file reasons still answer, with a named error, so a surprise call says what happened. The transfer code now has exactly two callers, `GetBytes` and `PutBytes`. Verified: a 256 KiB file copied in and back out through the stream path alone was byte-identical, at **one doorbell in each direction**. A 29-byte `*Type` became a single 1,024-byte block read.

The shape after the fix



Version 1: the wire speaks FileSwitch

The same day brought the transport change described in chapter 2. Version 1 commands carry the register frame itself. For `GetBytes`, that means:

Register	Meaning
R1	the filing system's handle
R2	the caller's buffer, as a guest logical address
R3	the byte count
R4	the file offset, so there is no separate seek

The module's transfer function collapsed accordingly (commit `475eb744d3`). The old loop split every transfer at 4 KiB, called `OS_Memory` per chunk to find a physical address, and issued a `SEEK` before each `READ`: two doorbells per page. A 256 KiB read went from **130 doorbells to 1**, five for the whole open, read and close. Now one request carries the frame, and chapter 4 covers how the host reaches the logical buffer:

```
req->cmd = (uint32_t)(is_write ? C_FS_PUTBYTES : C_FS_GETBYTES);
req->arglen = 0;
REQW(1) = h;
REQW(2) = mem;
REQW(3) = n;
REQW(4) = ptr;
rc = vmch_go(ws);           /* one doorbell */
...
if (rc == RC_BADADDR) {
    return REQW(3);         /* how far the translation got */
}
```

Built only partly

The version 1 design moves every FileSwitch entry onto register frames. The built code does so only where it paid:

Entry	Transport today
GetBytes, PutBytes	version 1 frames
File (read and write catalogue information)	version 1 frames
Func 30 and 35, Args 8	version 1 frames
Open, Close, most Args, directory listing, delete	version 0 commands

The design's own summary of this kind of state, elsewhere, is that the build is an honest hybrid. It works, and the remaining version 0 commands are candidates for moving rather than bugs.

Filling in FileSwitch's contract

Booting and the desktop exercise far more of the filing-system interface than copying files does. Each missing reason showed up as a named failure, and each was added when something needed it. Two stories from that work are worth telling.

Rename was answering the wrong question. `FSEntry_Func` reason 8 is Rename. The module answered reason 8 as *read disc name*, which is reason 11. So `*Rename` wrote the disc name over the caller's new-pathname string, renamed nothing, and FileSwitch reported *Bad rename*. The fix, in commit `4438a1b811`, moved the disc name to reason 11. That exposed a second fault: once the disc-name call worked, FileSwitch began putting the disc name into every path it passed, as `:HostFS.$.name`. So the host's path resolver learned to skip that prefix.

Listings had a ceiling and no order. A directory listing came back in a single reply, one 4 KiB page of 63 entries. A larger directory reported *directory too big*: Python's library directory, with 334 entries, could not be listed. And entries came back in the host's order, which on Apple's file system is a hash order. FileCore keeps catalogues sorted, so a share ran `!Boot`'s start-up tasks in a different order from the card they were copied from.

Version 2.02, commit `e7e6deffae`, fixes both in one place:

- the host collects the whole directory

- it sorts by the RISC OS leafname, folded case-insensitively as FileCore does
- it pages the result, with a continuation index in and a *more to come* flag out

Sorting is also what makes the page index a stable function of the directory, so the two fixes share their code. It was verified with a 200-entry directory created in scrambled order, and with a 3,126-file tree.

The contract as it stands

Entry	Reasons handled
Open, Close, GetBytes, PutBytes	all used
Args	0–7, 8 (write zeroes), 9, 10
File	1–4 write catalogue information; 5, 9 read it; 6 delete; 7 create; 8 create directory; 10 block size
Func	0, 1, 8 rename, 10 boot, 11 disc name, 14, 15, 19, 23, 24, 27, 30, 35, and others as no-ops

Version 2.01 added one more discipline: **every error names what failed**. A host answer the module did not expect used to fall through to *unsupported operation*. It now reads, for example, *HostFS: host I/O error (FS_GETBYTES: cmd 257, rc 10)*, spelt exactly as the trace spells it (commit [676e664065](#)).

4. Reaching guest memory

A filing system's hardest job is moving bytes into and out of the caller's buffer. Version 1 of HostFS made a bold choice about it: **the host reaches the buffer itself, by walking the guest's page tables**, and the guest module never translates an address. This chapter covers that choice, the puzzle it ran into — a transfer that stopped after exactly 252 bytes — the wrong turn taken on the way to the answer, and the guest-side fix. It ends with the protection caveats that come with letting a device write through the MMU.

The central move

The version 1 design states it without hedging: all guest addresses on the wire are **logical**, exactly as FileSwitch passed them to the module, and **the host translates**. QEMU has a precedent for this in semihosting, where a debug call names a guest buffer by its virtual address and QEMU reads it.

The consequences are what the design wanted:

- **One doorbell per FileSwitch call**, whatever the size or alignment of the buffer.
- **The guest cannot mistranslate an address**, because it no longer translates one.
- **A failed translation is an error, not a guess**. The version 0 path fell back to the identity mapping when a translation failed, and the design calls silent corruption the designed behaviour of that path.

How the host does it

The host's access routine borrows the method the blitter device had already proven for reading sprites out of guest virtual memory (commit 2daf256a55). It translates one page at a time with QEMU's debug translation, then maps and copies through the system address space:

```
while (len) {
    TranslateForDebugResult tres;
    uint64_t page = addr & ~(uint64_t)0xFFF;
    uint32_t off = (uint32_t)(addr - page);
    uint32_t n = 0x1000 - off;
    hwaddr plen;
```

```
void *host;

if (n > len) {
    n = len;
}
if (!cpu_translate_for_debug(cpu, addr, &tres)) {
    vmch_trace("vmch: translate failed at va=%08llx\n",
               (unsigned long long)addr);
    vmch_dump_walk(cpu, addr);
    return false;
}
plen = n;
host = address_space_map(&address_space_memory, tres.physaddr, &plen,
                        is_write, tres.attrs);

...
if (is_write) {
    memcpy(host, p, n);
} else {
    memcpy(p, host, n);
}
address_space_unmap(&address_space_memory, host, plen, is_write,
                   is_write ? n : 0);

p += n;
addr += n;
len -= n;
}
```

QEMU's more obvious call for this, `cpu_memory_rw_debug`, was tried first. It routes through the CPU's own address space rather than the system one, and it reports any memory-transaction error as a translation failure. The rewrite avoids both.

A counted variant reports **how far it got**. A buffer is not necessarily reachable end to end, and nothing on the host can fault a page in. So a partly reachable buffer returns a short count with `RC_BADADDR`, and the count in R3, rather than quietly delivering rubbish past the boundary.

252 bytes, and not one more

The new transport worked at once for most transfers. `*Copy` of a 256 KiB file moved the whole file in one doorbell, from a buffer at `0x493aa000`: 64 pages, correct bytes. `*Type`, the Filer, and the C compiler working off HostFS all worked.

One case did not. A BASIC program DIMmed a 256 KiB buffer, which landed at `0x8F04` in application space, and read the file into it with `OS_GBPB`. **252 bytes transferred, and the page at 0x9000 would not translate.** 252 bytes is exactly the distance from `0x8F04` to the next page boundary.

Ruled out, by measurement

The investigation eliminated its suspects one at a time, and recorded each so nobody re-runs them (commits `049aae171c` and `2daf256a55`):

Suspect	Why it was ruled out
crossing a page boundary	a 12,320-byte transfer spanning four pages moved whole
the size of the transfer	the 256 KiB <code>*Copy</code> moved in one doorbell
the access path	rewritten on the blitter's proven pattern, it failed identically
QMP's address translation command as a witness	it said <i>unmapped</i> even for an address the same run had just transferred; it samples whatever task is current

The wrong turn

One test sent the investigation the wrong way. The obvious explanation was **lazy mapping**: RISC OS might not have mapped the buffer's pages yet. So the pages were touched from BASIC first, and the transfer still failed. The design record wrote down *it is not lazy mapping*, and moved on to the translation code. That bullet is still in the document, above the answer that contradicts it.

The answer

Commit [1dd95655a9](#) found it. **RISC OS maps application space lazily, and the debug walk was telling the truth the whole time.** The proof was one line of BASIC that touches the buffer's pages and then transfers, *in a single execution by the client itself*:

Touched first	Result
nothing	252 bytes; fails at 0x9000
every 4,096th byte	258,300 bytes; fails at 0x48000, the one page that stride misses
every page, last byte included	the whole 0x40000 bytes, one doorbell, correct

The failure address moved exactly as far as the touching did. The page tables were never wrong: an untouched page genuinely had no entry, and the debug walk reported that correctly. What made the difference was touching and transferring in the same execution; the record does not say why the earlier, separate touches did not hold.

One casualty is recorded honestly. During the hunt, a change to QEMU's page-table walker skipped a domain-fault check for debug translations. The commit that found the answer keeps it, and says plainly that it *demonstrably fixes nothing here, and can be dropped without loss*.

The fix: fault it in from the guest

The host cannot fault a page in, but the guest can. Commit [fe63582be5](#) adds the fix to the module's transfer function. It tries the single doorbell. On a short count caused by an unmapped buffer, it touches the buffer and tries once more:

```
static void touch_pages(uint32_t mem, uint32_t n)
{
    volatile const uint8_t *p = (volatile const uint8_t *)mem;
    uint32_t i;
    uint8_t sink = 0;

    if (n == 0) {
        return;
    }
}
```

```

    }
    for (i = 0; i < n; i += 0x1000u) {
        sink = (uint8_t)(sink | p[i]);
    }
    sink = (uint8_t)(sink | p[n - 1]); /* the final partial page */
    (void)sink;
}

static uint32_t xfer(struct hostws *ws, uint32_t h, uint32_t ptr,
                    uint32_t mem, uint32_t n, int is_write)
{
    uint32_t moved = xfer_once(ws, h, ptr, mem, n, is_write);

    if (moved != n && ws->w_xfer_rc == RC_BADADDR) {
        touch_pages(mem, n);
        moved = xfer_once(ws, h, ptr, mem, n, is_write);
    }
    return moved;
}

```

Each design choice has a reason:

- **It reads a byte per page, not writes one.** The touch goes through the guest's own abort handler, which maps the page, just as any other filing system's write into the buffer would. Reading leaves the buffer undisturbed if the transfer then fails.
- **It includes the last byte,** because the buffer's final partial page is exactly what a stride of 4,096 misses.
- **It retries once.** After touching, every page is present. The common case never reaches the retry, so the fast path stays at one doorbell.
- **It retries only an unmapped buffer.** Since version 2.01, any other refusal is reported at once, because it would only be refused again.

Measured on a fresh, untouched 256 KiB DIM: the first attempt moved 252 bytes, and the retry moved the whole 0x40000, with the first, middle and last bytes matching the host file.

Why it is safe here

The module's source explains why touching the caller's buffer is safe on this path. In paraphrase of the programmer's reference: for the block-transfer calls, FileSwitch knows the byte count and validates the buffer, whereas for certain `Func` reasons it cannot. So a wild pointer should never reach the touch loop, only an in-range page the client has not yet faulted in. If one ever did, the loop would take a data abort. The source names the fix, a validation call before the loop, which those `Func` entries must make for themselves.

Errors must never be silent

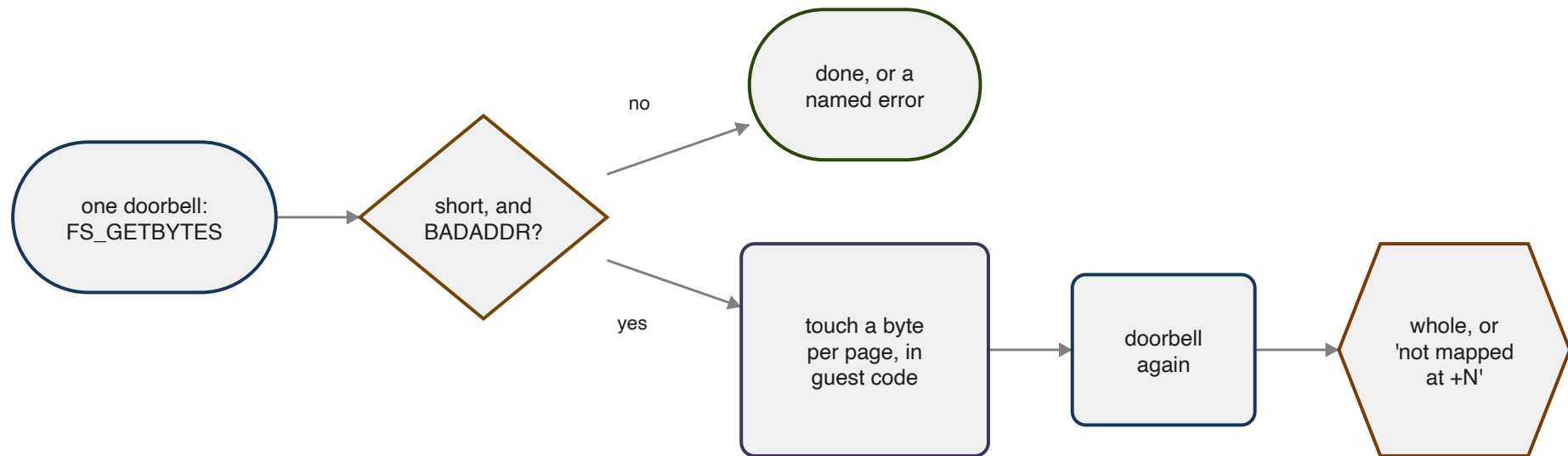
Buffered `GetBytes` has no exit registers. So a short transfer that is not reported is reported as success, and the module learned that the hard way. Before commit `27c5d2f124`, an `OS_GBPB` reported *0 bytes not transferred* over a buffer holding 252 good bytes and 261,892 stale ones.

Now any short transfer returns an error that names what happened:

```
if (moved != n) {
    return (ws->w_xfer_rc == RC_BADADDR) ? badaddr_at(ws, moved)
        : rc_to_error(ws, ws->w_xfer_rc);
}
```

The unmapped-buffer error carries the offset of the first unreachable byte, such as *transfer buffer not mapped at +258300*. The source comment gives the reason: a short transfer is nearly always a partial-coverage bug in whatever prepared the buffer, and the offset points straight at it. A bare *not mapped* would have left that to a diagnostic build.

The retry path



A staging design was drawn as well: copy through a buffer in the module area, which always translates, then `memcpy` in guest code. It was never built, because the touch made it unnecessary.

Caveats that come with the design

Letting a device write through the guest's MMU has consequences the project records, and a few that reading the code suggests.

Stated by the project:

- **It relies on TCG.** Writing behind the CPU's back through debug translation is fine under TCG, and the design notes it needs revisiting under hardware virtualisation.

- **The share root is the only perimeter.** Any guest code can ring the doorbell. The design advises saying so in user documentation, since sharing a whole home directory would be a poor choice.

INFERRED, from the code, and untested:

- **Protection is not checked.** QEMU's debug translation applies no access-permission checks, and the walker change above skipped domain checks too. So a client could ask HostFS to read file data into pages it could not write itself. On this machine the ROM image is copied into RAM, so that includes the ROM's own pages.
- **Snapshots and open files.** The first design called snapshots free, because the device keeps no state between requests. But the device holds host file descriptors, and it has no migration state. A `loadvm` into a fresh QEMU process with files open would find those handles dead. No result of the planned snapshot test is recorded.

5. Names, types and dates

A RISC OS file and a host file disagree about almost everything a name carries. RISC OS separates path components with a dot, and has no file extensions. Instead each file carries a 12-bit *file type*, stored in its load address along with a five-byte datestamp. It uses an 8-bit Acorn Latin-1 character set. A Mac or PC uses dots in names, extensions for type, UTF-8, and forbids characters RISC OS allows.

Version 1 moved every one of these translations onto the host. This chapter covers the rules as they now stand, and the bugs that shaped them, most of which appeared only when a real disc's worth of files was copied through.

The dot is a slash

RISC OS already has a convention for foreign names, used by its DOS filing system: a dot in a foreign leafname appears as `/`. So `readme.txt` on the host lists in RISC OS as `readme/txt`. HostFS follows it. `host_name_of` turns each `/` into `.` on the way out, and the reverse mapping turns dots into slashes on the way in.

The type rides the name

A host file's RISC OS type comes from its name, in this order:

1. **An explicit,xxx suffix**, three hex digits, gives the type, and the suffix is consumed. `hello,ffb` lists as `hello`, of type BASIC.
2. **Otherwise the extension** is looked up in a type table.
3. **Otherwise** the type is &FFF, Text. Directories take no type.

The host side of rule 1 had a subtle bug. The suffix was consumed, but what was left could still contain a dot, as in `readme.txt,ff9`, and the dot was returned untranslated. So the file listed as `readme.txt`, visible but impossible to open. The fix passes the remainder through the same name translation:

```
if (comma && strlen(comma + 1) == 3) {
    char *end;
    long t = strtol(comma + 1, &end, 16);
```

```
    if (end == comma + 4 && t >= 0 && t < 0x1000) {
        *type = is_dir ? 0 : (uint32_t)t;
        return guest_name_of(host_leaf, (size_t)(comma - host_leaf));
    }
}

out = guest_name_of(host_leaf, strlen(host_leaf));
if (is_dir) {
    *type = 0;
    return out;
}
dot = strrchr(host_leaf, '.');
*type = (dot && dot != host_leaf) ? type_for_ext(dot + 1) : 0xFFF;
return out;
```

Load, execute, and a datestamp

RISC OS stores a typed, dated file's type and time in two 32-bit words. The top twelve bits of the load word are all ones, the next twelve are the type, and the low byte plus the whole execute word hold a five-byte count of centiseconds since 1900. The host computes both words:

```
static void load_exec_for(uint32_t type, uint64_t cs,
                          uint32_t *load, uint32_t *exec)
{
    *load = 0xFFF00000u | ((type & 0xFFFu) << 8) | (uint32_t)((cs >> 32) & 0xFF);
    *exec = (uint32_t)cs;
}
```

Moving this to the host fixed a visible bug. The module's own version had special-cased type &FFF to all ones, so **every file in a *Ex listing came back untyped and undated**.

A type table, generated

The extension table is generated, not typed by hand. `tools/mktypemap.py` scrapes the RISC OS Open wiki's file-types list and keeps its extension column. The result is 179 entries: 156 extensions from 1,216 types, plus a local block of text extensions.

Inference must never produce an executable type

The generator drops every **executable type**: Module, Absolute, Utility, Obey, BASIC and Command. The design explains why. Those types mean *run me* when double-clicked, and guessing from an extension is not grounds for telling the desktop that an arbitrary host file is code. Only an explicit `,xxx` suffix may yield them. The design gives an example: mapping `.mod` to Module would hand a tracker music file to the module loader, *a crash at best*.

Two gaps between the table and its use

Reading the code shows two places where the table and its documentation part company:

- **.mod is in the table**, as type &CB6, the SoundTracker module format from the same wiki list. That is the right answer: a music file, not code. But the design document still says `.mod` is simply not in the table.
- **The generated table is not loaded by default**. Only a `VMCH_TYEMAP` environment variable loads it, and no launcher sets it. So a normal launch uses a **23-entry built-in table**, a fact the private user documentation also records. The generator's docstring mentions a device property for the table that does not exist.

INFERRED: the device does not re-check entries loaded from a table file, so a hand-edited line mapping `bas` to BASIC would be honoured, bypassing the executable rule.

Writing types back: *SetType renames

A type set in RISC OS has to survive on the host, and the host's only place to keep it is the name. So on HostFS, **setting a file's type renames the host file**. The policy, called `naming=smart` in the design, is spelt out in the source with examples:

```
* "hello/c", &FFF -> hello.c      the slash was a dot all along
* "notes",      &FFF -> notes      Text is the default: nothing to say
* "shot/png",   &B60 -> shot.png    the extension already says PNG
* "shot",       &B60 -> shot,b60    nothing says PNG, so the suffix does
* "logo",       &FF9 -> logo,ff9
* "data/png",   &FFF -> data.png,fff the extension says the wrong thing
```

A type is encoded only when it has to be. Text gets no decoration, because most files are text and a share full of `,fff` would be unusable from the host side. A name that already carries the right extension is left alone, so `hello.c` copies out as `hello.c`.

The rule itself is short:

```
static char *host_leaf_for(const char *host_base, uint32_t type)
{
    const char *dot = strrchr(host_base, '.');
    uint32_t implied = (dot && dot != host_base) ? type_for_ext(dot + 1)
                                                    : 0xFFFF;

    if (type == 0 || type == implied) {
        return g_strdup(host_base);    /* a directory, or already says it */
    }
    return g_strdup_printf("%s,%03x", host_base, type);
}
```

The bugs that shaped the rule

Both came from copying a real disc through HostFS (commit `569d875f8a`):

- **Typed files that could not be reopened.** An earlier rule *appended the table's extension*. A RISC OS file `SDIM0019` of type JPEG was created on the host as `SDIM0019.jpg`, which reads back as `SDIM0019/jpg`. `*Copy` creates a file and then opens it by the name it asked for, so every copied file with a type in the table failed with *file not found*. The rule now adds only `,xxx`, and only when the name would otherwise imply a different type.

- **Directories renamed ,ffd.** *Copy writes a directory's catalogue information too, and the rename rule gave every directory a suffix. Directories now take no type, date or attribute changes.

The result was verified the practical way. A module copied out of RISC OS arrives on the host as HostFSv6, ffa, and loads again with RMLoad without anyone setting its type.

Character sets

RISC OS names are 8-bit Acorn Latin-1. Apple's file system refuses names that are not valid UTF-8. The card's own tree includes a *Beginners Guide* folder spelt with hard spaces. Commit 569d875f8a maps names both ways:

RISC OS	Host
ASCII	unchanged
&A0, the hard space	a plain space
&80–&9F, Acorn's additions	a 32-entry table to Unicode
&A1–&FF	Latin-1 as UTF-8, normalised to NFC
anything unmappable, host to guest	shown as _, and still opens

Names the host cannot store

Windows forbids characters a RISC OS name may hold: < > " | ? *, a trailing dot or space, and device names such as NUL and CON. A real card tree copied cleanly on the Mac, and stopped partway on Windows. The trigger was a name, TRUE>>>1, inside the StrongED editor's application tree.

Commit 7dd24ec446 maps each such character to **U+F000 plus the character**, in Unicode's private-use area, and maps the whole U+F000–F0FF range back on the way in. This is the same spelling WSL and Cygwin use. The private-use area is chosen because a code point there cannot occur in an 8-bit Acorn name, so the mapping is total and needs no escape character. The guest never sees it.

The mapping is applied unconditionally, even on hosts that would accept the name. The commit explains why: the two cases that matter raise no error at all on NTFS. An untyped file named `NUL` opens as the null device and swallows the data. A trailing dot is silently dropped, so `foo/`, which is `foo.` on the host, collides with `foo`. Only counting files catches either failure, which is why the tree-building tool in chapter 7 counts.

```
if (end > 0 && (ideal[end - 1] == '.' || ideal[end - 1] == ' ')) {
    trail = (unsigned char)ideal[end - 1];    /* Windows drops these */
    end--;
}
if (win32_reserved(ideal, end)) {
    g_string_append_unichar(out, 0xF000 + (unsigned char)ideal[0]);
    i = 1;                                   /* break the device name */
}
for (; i < end; i++) {
    unsigned char c = (unsigned char)ideal[i];

    if (c == '<' || c == '>' || c == '"' || c == '|'
        || c == '?' || c == '*') {
        g_string_append_unichar(out, 0xF000 + c);
    } else {
        g_string_append_c(out, (char)c);    /* ASCII or UTF-8 byte */
    }
}
if (trail) {
    g_string_append_unichar(out, 0xF000 + trail);
}
```

Case

RISC OS names are case-insensitive. Most host file systems are case-sensitive, or preserve case. Each path component is resolved with an exact `stat` first, then, if that misses, a case-insensitive scan of the directory, comparing each entry's *RISC OS* name. A name that is not found at all resolves to its natural host spelling, ready for a create.

Dates, and a question still open

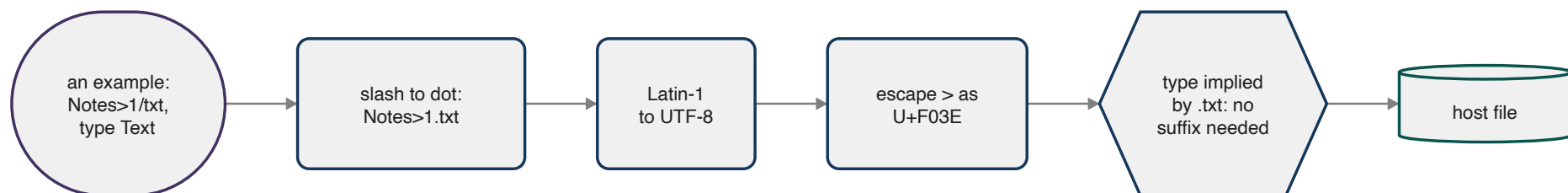
RISC OS timestamps have no time zone. The host side converts a file's modification time to centiseconds since 1900, and commit 174a719248 added the host's UTC offset while doing so. The reasoning at the time, from the source comment: RISC OS keeps local time in a filestamp, so without the offset every file read an hour early under British Summer Time. It was verified: a file saved at 15:22:20 BST listed correctly.

A later section of the same design reverses that reasoning. RISC OS timestamps are **UTC by definition**, it says. The guest's clock runs in UTC with no zone configured, and the host maps timestamps as local time, so a file saved from the guest reads an hour old on a Mac in BST. The right fix, in that section's words, is *UTC on the wire and the zone configured in the guest, not a different offset in the host*. **The code has not changed**, and the question is listed as open.

Attributes

Only owner-write is read from the host: a read-only host file lists as not writable. On the way back, RISC OS's *writable and not locked* maps to the owner-write bit. Dates written from RISC OS are applied with `utime`. A tree copied through HostFS keeps card dates from 1996 and 2012.

A name's journey



6. Into the ROM

Until 12 September, HostFS was loaded the way RISC OS loads most extensions: from the SD card's boot sequence, after the machine had started. To boot *from* the share, that cannot work. This chapter covers why, the two designs for getting the module into a stock ROM, the one that was built, and what a module must give up to run from ROM. That last part cost a day, and changed how every result is now tested.

A filing system cannot load itself

The boot design puts the constraint first:

Any scheme that loads the module from a filesystem in order to be that filesystem is a trap, and it fails late and confusingly rather than cleanly.

The boot filing system has to exist during the kernel's own module scan, before anything looks for `!Boot`. That rules out loading HostFS from the card's start-up sequence, and loading it with `*RMLoad` from anywhere. The third option, building a ROM from the RISC OS sources, works, and it is what some emulator setups do. **It was ruled out from the start:** the project runs the stock RISC OS Open ROM.

That leaves one route: put HostFS into the stock ROM image's list of modules, so the kernel initialises it exactly as it initialises its own filing systems.

The module chain

A RISC OS ROM holds its modules on a **chain**. It starts at a known place in the kernel. Each module is preceded by a size word, which holds that module's length plus the next size word, and a zero word ends the chain. The kernel walks it that way at start-up, with no checksum and no bounds check.

That makes one operation easy: **append at the terminator**. The splicer's docstring spells out why appending is also correct. Chain order is initialisation order, so a module appended at the end initialises after every stock module, FileSwitch included, which is what a filing system needs.

Designed, and built

The boot design and the tool that was built differ on two points:

	Designed	Built: <code>tools/mkrom.py</code>
where	inside QEMU, spliced as the ROM loads, through a device property	a host-side tool, run by the launch scripts
image size	grown, with the header's size updated so the CMOS address moves with it	not grown : new modules go into the slack at the end
headroom	about 64 KiB assumed	268 KiB of slack measured on the 5.31 image

The design had mentioned a host-side tool only as an escape hatch. It became the answer, and no in-QEMU injector exists.

How the splice avoids growing the image

Between the chain's terminator and the footers at the end of the ROM, there is slack: the credits text, then padding of `0xFF` bytes. The splicer writes the new modules over the terminator, moves the credits forward by the same amount, and lets the padding absorb the difference:

```
insert = b''.join(struct.pack('<I', len(b) + 4) + b for _, b in bodies) \
    + struct.pack('<I', 0)
needed = len(insert) - 4          # net growth inside the slack
if needed > free:
    raise SystemExit(f'{needed} bytes needed, {free} of slack: '
                    'image would have to grow — not supported')

out = bytearray()
out += base[:term - 4]           # everything up to the old zero word
out += insert                    # our modules, then a new zero word
out += base[term:shift_end]      # credits, moved forward
out += b'\xFF' * (tail - len(out)) # padding absorbs the rest
out += base[tail:]               # footers, byte-for-byte, same place
assert len(out) == len(base), 'image size changed'
```

No header field is touched. The docstring gives the reasoning. The kernel finds the footers from a constant fixed at build time, not from the image header. The HAL reads the image size only to find the CMOS blob and to copy the ROM into RAM. So with the size unchanged, everything that reads the image stays correct.

The docstring also records what growing the image would take, in case the slack ever runs out. Both the image size and the *compressed* size in the header would need updating, because the HAL copies by the compressed size, and the CMOS blob would need re-placing. That is deliberately not implemented. The tool refuses instead.

A launch log shows the result: *chain 137 -> 139 modules ... image size unchanged at 0x500000*. The launch scripts cache the spliced image under a hash of the stock ROM and every module's contents, so editing a module rebuilds it and a relaunch does not.

The first splice crashed, and not where expected

The plan for ROM safety looked simple. A module in ROM executes in place, from read-only memory, so it must not write to its own image. So HostFS's variables moved into one structure claimed from the RMA, the module area, with its address kept in the module's private word. The plan also assumed the C compiler's module option would handle the rest, and added a note: *verify this before trusting it*.

Three writes into the image

The first spliced boot died with *Abort on data transfer at &FC4BFC8C*. The design record traced that address, and it was not in HostFS's code at all. It found three separate ways the conventionally built module wrote into its own image, each read in the source and confirmed in the binary (commit `bb79635c18`):

Culprit	What it does at initialisation
the linker's relocation code	the cmhg header's init calls <code>__RelocCode</code> , which adds the load address to every absolute word in the module: 58 of them. The faulting PC was the STR in that routine
the shared C library stubs	initialising them writes branch instructions over the stub vectors, inside the image
the C runtime	stores its own block in the private word, which every cmhg veneer then loads from, so HostFS's plan to use that word would have broken every entry

An earlier claim fell with it. Commit `cbee066699` had presented evidence that the module was ROM-safe. That evidence came from a **soft-loaded** module, where the C library copies the variables into the RMA anyway, so it proved nothing about ROM.

The recipe that runs in place

The module is still built entirely with the RISC OS Open DDE, but in a way that writes nothing into the image. The build script is short, and its comment names the rule:

```
| No cmhg and no C:o.stubs.  cmhg's header imports __RelocCode, and the
| stubs patch their vectors in place: both write into the module image at
| initialisation, which from ROM is a data abort (see c.hostfs, s.head).
| o.relocs is decaof's list of relocations: it must hold only PC-relative
| ones, since an absolute address would be wrong wherever the module lands.
objasm -o o.head s.head
cc -c -zps1 -ffah -o o.hostfs c.hostfs
link -rmf -o HostFS,ffa o.head o.hostfs
```

```
decaof -r o.head o.hostfs { > o.relocs }
```

Step by step:

1. **objasm assembles s.head:** the module header, the entry veneers, a `swix` routine that calls any SWI, and the constant data C would otherwise reach through an absolute address — the filing-system information block and the error blocks, every word an offset.
2. **cc -zps1 compiles the C** with no statics and no C library. That means no `printf` and no division, which the compiler turns into a library call. String literals are reached PC-relative.
3. **link -rmf links just those two objects.** The linker appends relocation code only to a module that imports it, and nothing does.
4. **decaof -r must show only PC-relative relocations.** At 2.00 there were 10 in the header and 19 in the C, all branches.

Everything writable, in one structure

The C source makes the rule impossible to break by accident. There is no file-scope pointer to the workspace, because that pointer would itself be a writable static. Every handler receives the workspace from the private word, and every helper takes it as an argument. Macros keep the old names readable, and **require a ws in scope**, so code that was not given the workspace fails to compile:

```
struct hostws {
    uint32_t w_freesave[8];           /* FreeEntry's registers, in s.head:
                                     * it must be first */
    volatile uint32_t *w_vmch;        /* doorbell, from OS_Memory 13 */
    struct vmreq *w_req;              /* page-aligned inside w_reqstore */
    uint32_t w_req_phys;              /* its physical address */
    ...
    uint8_t w_reqstore[sizeof(struct vmreq) + 4096];
};

#define WS_OF(pw)      (*(struct hostws **)(pw))

#define vmch           (ws->w_vmch)
#define req            (ws->w_req)
```

The structure is claimed with `OS_Module 6` — size in **R3**, the same register trap the GVFill module met — and zeroed by hand, because the RMA is not guaranteed clean.

Smaller, and verified

Build	Image size
1.01, cmhg and C library stubs	12,260 bytes
version 1 streams, still with stubs	12,644 bytes
2.00, ROM-safe	7,632 bytes
2.01, every error named	10,228 bytes
2.02, paged and sorted listings	10,300 bytes

Verified on 12 September with a stock ROM spliced with 2.00, and nothing soft-loaded: `*Modules` lists HostFS at `&FC4BCDA8`, a ROM address. Its 16,660-byte workspace is in the RMA. `*Ex HostFS:` lists real types and dates, and a 256 KiB copy in and back out is identical. The same image also soft-loads with `RMLoad` and passes the same tests.

One bug fell out on the way. `hostfs_final` rang the doorbell to close files even when the doorbell had never been mapped, so killing an unused HostFS wrote near address zero. The lazy mapping from chapter 2 turned out to be load-bearing too: initialisation from the ROM chain runs even earlier than a soft-load.

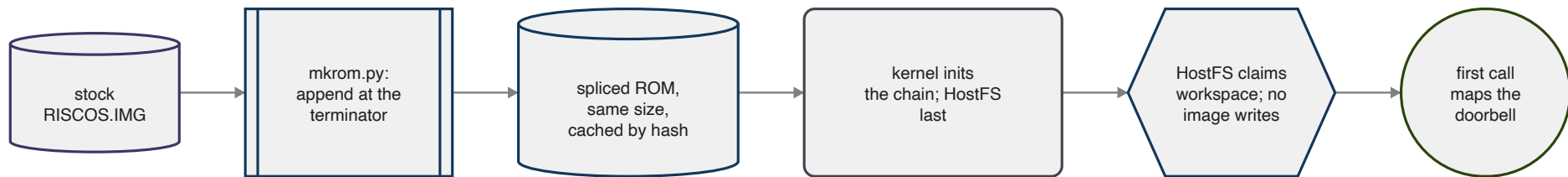
A new acceptance rule

The crash that `*Modules` could not see changed how every later result is judged. The design now states it at the top of its remaining sprints:

A result counts only when it works from a spliced ROM with nothing soft-loaded. `*Modules` showing a ROM address proves the module loaded onto the chain, not that it runs — a module can splice, list, and then abort on its first write.

A related warning is recorded about a parallel branch. `zcode/romloader`'s commit `f5abc37686` converted the old module to run from ROM, but kept `cmhg` and the stubs, kept a static workspace pointer, and passed the claim size in R2. The design record says it should not be merged, because the relocation code would still abort before any of it ran.

From stock ROM to a running filing system



7. Booting from a directory

With HostFS in the ROM, the last step was to make it the disc the machine starts from. The boot design describes the goal in one sentence:

The card stops being the root of the world and becomes optional baggage.

This chapter walks through the pieces that achieved it on 13 September: the launch scripts, the kernel's boot call, CMOS settings kept on the share, modules loaded from it, and the tree of files that makes a share bootable. It also covers the traps found on the way.

The launch scripts

On the Mac, `tools/rom.zsh` is sourced by every launcher. Two environment variables drive it:

Variable	Effect
RISCOS_HOSTFS=<dir>	the share, and HostFS plus its icon-bar filer spliced into the ROM
RISCOS_BOOT=hostfs	a CMOS blob that configures HostFS as the boot filing system

The splice logic puts the two HostFS modules ahead of any other modules requested, unless a module with the same title was already named:

```
if [[ -n "${RISCOS_HOSTFS:-}" ]]; then
  local list="${here:h}/hostfs/dde/HostFS,ffa ${here:h}/hostfs/filer/HostFSfiler,ffa"
  list="${RISCOS_HOSTFS_MODULES-$list}"
  hostfs_mods=("${list}")
  ...
  for hm in $hostfs_mods; do
    ...
    title="$_rom_module_title "$hm""
    (( ${titles[(Ie)$title]} )) || add+=("$hm")
  done
  mods=($add $mods)
```

```
fi
```

The design gives the reason for splicing whenever there is a share: no machine should start with a share and no filing system to reach it. On Windows, `tools/rom.py` carries the same logic for the `run.py` and farm launchers, as `--hostfs` and `--boot hostfs`.

CMOS that says *boot from HostFS*

The boot filing system is a CMOS setting. So `RISCOS_BOOT=hostfs` builds a blob with the CMOS tool from the QEMU walkthrough, setting filing system 220:

```
[[ -e "$RISCOS_HOSTFS/CMOS,ff2" ]] && saved="$RISCOS_HOSTFS/CMOS,ff2"
...
base="${saved:-$CMOS}"
out="$images/cmos-hostfs.bin"
# Remade every launch: it is cheap, and the share may have changed.
# 220 is HostFS's filing system number (hostfs/dde/s.head).
python3 "$here/mkcmos.py" --symbols "$here/cmos-symbols-530.json" \
    --base "$base" --filesystem 220 -o "$out" 2>/dev/null || { ... }
if [[ -n "$saved" ]]; then
    print "cmos: from the share's ${saved:t}"
else
    cp "$out" "$RISCOS_HOSTFS/CMOS,ff2"
    print "cmos: FileSystem HostFS; the share now keeps it, as CMOS,ff2"
fi
```

If the share already holds a `CMOS,ff2` file, that is the starting point. Otherwise the blob is seeded, and the share keeps a copy from then on.

How the kernel boots a filing system

The design record confirmed the kernel's path in the RISC OS sources before building anything:

1. At the end of start-up, unless Shift is held or the machine is set not to boot, the kernel calls `OS_FSControl 15`.

2. FileSwitch passes that to the **configured** filing system's `FSEntry_Func` reason 10.
3. That filing system performs its boot action.

FileCore's boot action, for boot option 2, runs `&.!Boot` — the `!Boot` application at the root of the user's disc. HostFS does the same when the share has one:

```
case 10:                                /* boot filing system */
    if (boot_option(ws) == 2) {
        _kernel_swi_regs cli;

        cli.r[0] = (int)"Run &.!Boot";
        return swix(XOS_CL_I, &cli, &cli);
    }
    return NULL;
```

`boot_option` answers 2 when `$.!Boot` exists and 0 when it does not. There is no disc record to hold any other value, so `*Opt 4` changes nothing. A share with no `!Boot` quietly boots nothing, as option 0 would.

A real disc on the icon bar

HostFSFiler 2.00 gives HostFS an icon on the icon bar, left of the discs, written on the pattern of RISC OS's RAM filing system's filer and also run from the ROM. Clicking it opens the share in a Filer window with every file's real icon. Its menu offers *Free*, which opens RISC OS's free-space window on the host volume, reported through the module's free-space entries: 954 GB, of which 544 GB free, on the machine that measured it.

By mouse, a new directory, drag-copies both ways, *Info*, rename and delete were each checked on the host.

Modules on the share, not in the ROM

Only the filing system has to be in the ROM, because only the filing system has to load itself. Every other module can be an ordinary soft-load from a disc that is already there. So the decision recorded on 13 September is that **the ROM carries HostFS and its filer, and nothing else**. Everything else lives in

the share's `$.Modules` directory, loaded before the desktop by one line in the boot sequence:

```
| HostModules - load the modules kept on the HostFS share, before the desktop.
| HostFS itself is in the ROM, so the share is reachable by now; everything
| else lives in HostFS:$.Modules, where it is easy to add, update or remove.
| -Sort loads them in name order; -Continue keeps one that fails from
| stopping the rest (the first error is left in X$Error).
IfThere HostFS:$.Modules Then Repeat RMLoad HostFS:$.Modules -Type &FFA -Sort -Continue
```

Adding, updating or removing a module is copying a file on the host, with no ROM to re-splice and no need to be ROM-safe.

The blitter module showed this is more than tidiness. Spliced into the ROM, GVFill never sees a sprite plot, because something later in the boot claims the sprite vector in front of it. Loaded from the share, it sees every plot. The [graphics walkthrough](#) has that story.

The loading line was tested against a failure. With a non-module named `Aardvark,ffa` ahead of GVFill, GVFill still loaded, `X$Error` read *Illegal header field in module*, and the desktop was unchanged.

CMOS that persists

A real Pi keeps its CMOS settings in a file on the SD card. A module called SDCMOS saves them there after every change, and unloads itself when there is no card. So a machine booted from HostFS kept nothing: every `*Configure` was lost at power-off.

HostFS now does SDCMOS's job for the share (commit `509ab9d478`). After start-up, a callback checks two things:

```
r.r[0] = 161; /* OS_Byte 161: read CMOS */
r.r[1] = 5; /* FileLangCMOS: *Configure FileSystem */
if (swix(XOS_Byte, &r, &r) != NULL || r.r[2] != 220) {
    return;
}
r.r[0] = 23; /* OS_File 23: info with type */
r.r[1] = (int)"HostFS:$.CMOS";
if (swix(XOS_File, &r, &r) != NULL || r.r[0] != 1
    || (r.r[6] != 0xFF2 && r.r[6] != 0xFE4)) {
```

```

    return;
}
if (hostfs_cmos_hooks(pw, CMOS_BYTEV_CLAIM) == NULL) {
    ws->w_cmos_saving = 1;
}

```

Only if HostFS is the configured filing system **and** the share holds \$.CMOS of a configuration type does HostFS claim the CMOS write vector. It then runs *SaveCMOS HostFS:\$.CMOS after every CMOS write, writing the same 2,052-byte blob the HAL loads. A share that is not the boot disc is never written to. The vector and callback addresses come from ADR in the assembly header, because a function address taken in C would be an absolute relocation.

Verified: *Configure Delay 20 rewrote CMOS, ff2 within a second. After power-off and relaunch, *Status Delay said 20.

Making a share bootable

A bootable share is a copy of a card's tree. The first was made by hand, inside a machine with both the card and the share attached. Seven top-level directories — !Boot, Apps, Documents, Utilities, Printing, Public, Diversions — were copied with *Copy: **5,229 files, 320 MB, in 65 seconds**. One start-up file had to come out: it points at the card's development tools and fails without the card.

The by-hand copy took a morning. Most of it went on not knowing that a recursive *Copy had stopped five levels down, leaving the result 40% short while looking finished. tools/mktree.py automates the sequence with the checking that was missing (commit 624275c26c). For each entry, it counts the entry on the card, copies it, counts what landed on the host, and descends a level wherever the two disagree. That isolates the one entry that will not copy instead of losing it and everything after it.

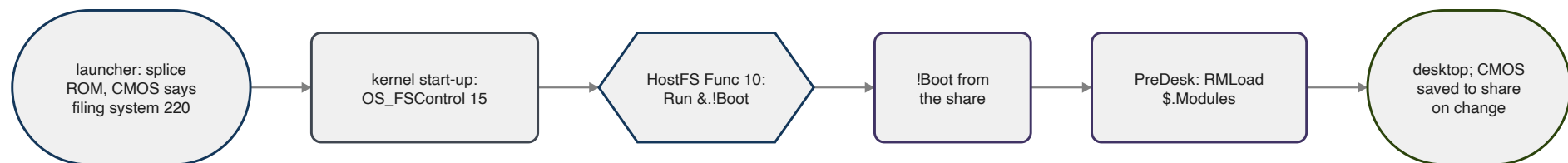
The tool records two rules learned the hard way:

- **Never count a HostFS directory from the guest.** At the time, a listing stopped at 63 entries, so a complete 762-file directory counted as 364. The card is counted over SDFS, and the host by walking its own file system. Version 2.02 has since removed the limit.
- **Never trust the absence of an error.** Two name failures on NTFS raise nothing at all (chapter 5). Only counting files catches them.

Traps worth knowing

Trap	What happens	The answer
a card shadows the ROM	a card's start-up file loads its own HostFS 1.01 unconditionally, and RMLoad replaces the ROM's 2.00	make that line <code>RMEnsure HostFS 2.00</code> ; the shared card images still need the change
a snapshot decides its own ROM	<code>-loadvm</code> restores RAM, and the ROM lives in RAM, so a machine saved without HostFS in ROM comes back without it	the Windows launcher refuses a HostFS boot with a snapshot
pull, then rebuild	a 2.00 module on an emulator binary older than the naming work answers <code>*Cat HostFS: with <i>bad path</i></code> , which looks like a guest fault	rebuild the emulator, not just the module

The boot, end to end



With no card attached, a machine booted this way reached a clean desktop — the Raspberry Pi backdrop, and the pinboard's NetSurf and StrongED, all from the share — in 13.5 to 14.3 seconds after launch. `Boot$Dir` is on HostFS, the scrap directory is on the share, and the boot's own modules load from it. On the Windows test farm, an instance booted with no card reached the desktop in 33 seconds, with the C compiler working from the share.

8. Measured, limited, unfinished

This chapter gathers the measurements, then what is still open: first as the project states it, then as found by reading the code. It ends with the question the project raised about what could ever be shipped.

SD card against HostFS

The boot design predicted HostFS would beat the emulated SD card substantially, and gave the reason as crossings. Every register access to the emulated SD controller traps into QEMU, and a 512-byte sector through a FIFO is about 128 of them, before FileCore's own work. HostFS is one crossing per filing-system operation.

The boot, timed

The measurement used the same ROM and the same tree, with the card's extra start-up file removed so that both booted identical !Boot sequences. It was run on the Mac under TCG, to a settled desktop:

	SD card	HostFS
wall clock, three runs	16.3, 15.1, 15.8 s: 15.7 s	14.0, 14.5, 14.8 s: 14.4 s
host crossings	2,434,544 SD controller register accesses	2,657 doorbells
of which data	2,351,650 reads of the data port	311 reads, whole
device commands	827, including 730 multi-block reads	—
host time in the device	not measured	0.103 s in all: 39 µs a request

HostFS is faster by 1.3 seconds, 8%. The design record explains why nine hundred times fewer crossings buy so little:

the boot is bound by the emulated CPU, and under TCG an MMIO access is a function call, not an exit, so nine hundred times fewer crossings buy little. Under hardware virtualisation each of those 2.4 million accesses would be a VM exit, and the gap would be the design's argument; under TCG it is a footnote.

Where the doorbells go

One boot from the share, counted from the trace:

Operation	Doorbells
catalogue reads: <i>does this exist, and what is it?</i>	1,894
... of which misses, where the file was not there	722
data reads	315
opens and closes	203
directory listings	32
buffers faulted in and retried	4

The boot design had warned that existence checks would run into the thousands, and they do. A miss on a typed name scans the whole directory, because `notes` might be stored as `notes`, `ffb`. An index was planned before any boot attempt. Measured instead, **the 722 missing lookups cost 33 ms of a 14.8-second boot**, so the index stays unbuilt until a workload says otherwise.

Four ways to lose the win

The boot design listed the mistakes that would give the advantage away, and the build avoids all four:

Mistake	HostFS today
a host open, stat and close per RISC OS operation	one request per FileSwitch call
fsync on every write	none
a coherence stat on every open	none; the host file system is the truth
a doorbell per catalogue entry	a whole page of entries per doorbell

Design against build

Area	The design	Built
translation	the wire speaks FileSwitch	a hybrid of version 1 frames and version 0 commands
addresses	logical; the host walks the MMU	the request block physical; buffers logical
directories	listings cached with a time-to-live	no cache; paged and sorted per request
errors	a registered error base	named errors, on unregistered numbers
security	realpath containment, plus a readonly= option	containment built; no read-only option
guest module	about 250–300 lines	1,724 lines of C and 507 of assembly

The module is larger than planned mostly for honest reasons. ROM safety needed hand-built string routines and a `swix` in assembly. The desktop needed a filer, free space, directory creation and paging. And every error now says what failed.

Open, as the project states it

- **Filing system number 220 is unallocated**, and error numbers are ad hoc. Both need registering with RISC OS Open before anything ships outside the team.
- **No wildcards** in the calls that write catalogue information, and untyped files lose their load and execute addresses.
- **Time zones**: UTC on the wire, and the zone in the guest, is the stated fix, and the code still adds the host offset (chapter 5).
- ***Free from a HostFS directory** is FileCore's command, so it does not work there.
- **Not built**: a read-only option, several shares selected by disc name, zero-copy transfers.
- **The shared card images** still load HostFS 1.01 unconditionally over the ROM's copy.
- **The filer uses the hard-disc icon**.

Open, found by reading the code

Every item here is **INFERRED**: read from the code at `e7e6deffae`, and not tested.

A create through a symlink can leave the share

`host_path` resolves a guest path component by component under the share root. It then checks containment with `realpath`, so that a symlink pointing out of the share is refused. The check only applies when `realpath` succeeds:

```
char *real = realpath(cur, NULL);
char *rootreal = realpath(s->root, NULL);
bool ok = true;

if (real && rootreal) {
    size_t rl = strlen(rootreal);
    ok = strncmp(real, rootreal, rl) == 0 &&
        (real[rl] == '\0' || real[rl] == G_DIR_SEPARATOR);
}
```

```
}
```

`realpath` fails for a path that does not exist yet, so `ok` stays true. Suppose the share contains a symlinked directory pointing outside it. Then a **create or rename to a new name inside that directory** passes the check, and the host follows the link. Reading an existing file through the same link is refused correctly.

The guest cannot create symlinks itself, so this needs a link already present in the share. The design's own security section asks for symlinks to be resolved before the containment check. A fix would resolve the *parent* directory when the leaf does not exist yet.

The build of 12 September had a second route out. A path component of `//` became `..` on the host. Version 2.02's escaping of trailing dots, added for Windows, turns that into an ordinary name, and closes the route incidentally.

Found by reading the code, continued

Where	What
guest memory access	debug translation checks no page protection, so file data can be written into pages the client itself could not write, including the ROM's copy in RAM
snapshots	the device holds host file descriptors and has no migration state; a <code>loadvm</code> into a new process with files open would find them gone
directory listings	a host name of 40 bytes or more is silently left out

Found by reading the code, concluded

Where	What
names	notes and notes, ffb in one directory both list as notes
the version 1 header	the error-number and PSR words are defined, and never written
the trace	has no off switch

What could be shipped

The boot design ends with the question every emulator of RISC OS eventually meets: what could be given to someone else? It frames its answer as a summary of the licences, and says plainly that it is not legal advice:

- **The ROM** is Apache 2.0 for most of RISC OS, but not all of it. Shipping one needs an inventory of what is inside.
- **RISC OS Direct images must not be redistributed.** They carry commercial third-party applications.
- **Broadcom's firmware** carries a use restriction, but QEMU never runs it, so an emulator image can simply leave it out.
- **The name** "RISC OS" is a trademark. Describing software as running RISC OS is ordinary use; naming a product after it needs permission.
- **The recommendation:** build the boot tree from the Apache-licensed sources, and ask RISC OS Open and RISC OS Developments directly. As the design puts it, a written yes is worth more than any reading of a licence page.

One provenance note belongs to this document as well. The public design notes behind HostFS quote the RISC OS programmer's reference and RISC OS Open's filing-system guide closely, sentence by sentence, because that is how their claims stay checkable. Some module source comments do too. This document paraphrases every such passage, and quotes only the project's own words.